

Introduction to Python Packages

What is a Package?

- **Package:** In Python, a package is a collection of modules organised in a directory hierarchy. A package allows you to organise and structure your Python code in a modular way, making it easier to maintain and reuse.
- **Module:** A module is a single Python file that can contain functions, classes, and variables. It can be imported and used in other Python programs.
- **Standard Library and Third-Party Packages:** Python comes with a vast standard library of packages, and you can also install third-party packages via tools like **pip**.

Why Use Packages?

- **Modularity:** Packages help break down large programs into smaller, manageable modules.
- **Reusability:** Code can be reused across different programs by importing modules from packages.
- **Namespace Management:** Packages help avoid naming conflicts by organising code into separate namespaces.

Creating a Custom Python Package in Windows Subsystem for Linux (WSL)

- Open the Word document “*Creating custom package*”. It provides a step-by-step approach to creating a custom package on a Windows computer. It assumes you have already installed Windows Subsystem for Linux (WSL). (If you already have Linux, you can skip its installation)

Create a Directory: Create a directory that will contain your package modules.
Add an `__init__.py` File: Add an empty `__init__.py` file in the directory. This file signals to Python that the directory should be treated as a package. It can be empty or contain initialisation code for the package, such as import statements.

Add Modules: Create Python files (modules) inside the directory. Each file can contain related functions and classes.

Use the Package: Import the package or specific modules in your Python script and use the functionality provided.

Fancy Indexing

- Fancy indexing involves using arrays (or lists) of integers to index another array (*passing an array of indices to access multiple array elements at once.*) This can be very useful when you need to access multiple non-contiguous elements or specific patterns within an array.
- Allows you to create new arrays by specifying the exact indices of the elements you want to extract.
- Fancy indexing allows you to access and manipulate elements based on complex criteria that would be difficult to achieve with regular slicing.
- You can select multiple, non-contiguous elements from an array, which is useful for various advanced operations.

One-dimensional
output: [10, 30, 50]

```
import numpy as np

# Create a 1D numpy array
array = np.array([10, 20, 30, 40, 50])

# Use fancy indexing to access elements
# at indices 0, 2, and 4
indices = [0, 2, 4]
fancy_indexed_elements = array[indices]

print(fancy_indexed_elements)
```

```
import numpy as np

# Create a 2D numpy array
array_2d = np.array([[1, 2, 3],
                     [4, 5, 6],
                     [7, 8, 9]])

# Use fancy indexing to access elements at
# positions (0, 1), (1, 2), and (2, 0)
row_indices = [0, 1, 2]
col_indices = [1, 2, 0]
fancy_indexed_elements = array_2d[row_indices, col_indices]

print(fancy_indexed_elements)
```

Multi-dimensional arrays.
Output: [2, 6, 7]

Fancy Indexing

The number at row 2 and column 0 is 7

The number at row 0 and column 1 is 2

The number at row 1 and column 2 is 6

```
import numpy as np

# Create a 2D numpy array
array_2d = np.array([[1, 2, 3],
                     [4, 5, 6],
                     [7, 8, 9]])

# Use fancy indexing to access elements at
# positions (0, 1), (1, 2), and (2, 0)
row_indices = [0, 1, 2]
col_indices = [1, 2, 0]
fancy_indexed_elements = array_2d[row_indices, col_indices]

print(fancy_indexed_elements)
```

Row

column

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

Output: [2, 6, 7]

- Selecting elements based on certain conditions.
- Reordering or reshaping data.
- Creating **masks** or filters to process specific parts of an array.
- Fancy indexing is often used in conjunction with boolean arrays (**masks**) to select elements that meet specific conditions.

```
import numpy as np

# Create a numpy array
array = np.array([10, 15, 20, 25, 30])

# Create a boolean mask for elements greater than 20
mask = array > 20

# Use the mask with fancy indexing
filtered_elements = array[mask]

print(filtered_elements)
```

In this example, mask is a boolean array that is used to filter the elements of array that are greater than 20.

Output = [25, 30]

Masking in numpy

- The practice of using boolean arrays to select elements of another array.
- It is a boolean array that is the same shape as the original array, where each element is either **True** or **False**.
- When you apply this mask to the original array, you retrieve elements that correspond to **True** values in the mask.
- In brief, it allows to:
 - Select elements of an array based on conditions.
 - Modify elements that meet certain criteria.
 - Combine multiple conditions to create complex masks.

Masking is commonly used in data analysis for:

- Filtering data based on conditions.
- Cleaning or modifying specific parts of a dataset.
- Performing computations on subsets of data.
- Visualizing specific data points in plots.

Masking in numpy

```
import numpy as np

# Create a numpy array
array = np.array([10, 15, 20, 25, 30])

# Create a boolean mask for elements greater than 20
mask = array > 20

# Use the mask with fancy indexing
filtered_elements = array[mask]

print("Original Array:", array)
print("Mask:", mask)
print("Filtered Elements:", filtered_elements)
```

```
Original Array: [10 15 20 25 30]
Mask: [False False False True True]
Filtered Elements: [25 30]
```

array > 20 creates a boolean mask where elements greater than 20 are marked as True. Applying the mask **array[mask]** returns only the elements of **array** where the mask is **True**

```
import numpy as np

# Create a 2D numpy array
array_2d = np.array([[1, 2, 3],
                     [4, 5, 6],
                     [7, 8, 9]])

# Create a boolean mask for elements greater than 4
mask_2d = array_2d > 4

# Apply the mask to the 2D array
filtered_elements_2d = array_2d[mask_2d]

print("Original 2D Array:\n", array_2d)
print("2D Mask:\n", mask_2d)
print("Filtered Elements 2D:", filtered_elements_2d)
```

```
[[ 7  8]
 [11 12]]
[2 6 7]
Original Array: [10 15 20 25 30]
Mask: [False False False True True]
Filtered Elements: [25 30]
Original 2D Array:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
2D Mask:
[[False False False]
 [False True  True]
 [ True  True  True]]
Filtered Elements 2D: [5 6 7 8 9]
```

array_2d > 4 creates a boolean mask for the 2D array. Applying the mask **array_2d[mask_2d]** returns a 1D array containing elements of **array_2d** where the mask is **True**.

Modifying Arrays Using Masks

The mask **array <= 20** identifies elements less than or equal to 20.

array[mask] = 0 sets those elements to 0.

```
import numpy as np

# Create a numpy array
array = np.array([10, 15, 20, 25, 30])

# Create a boolean mask for elements less than or equal to 20
mask = array <= 20

# Modify the array using the mask
array[mask] = 0

print("Modified Array:", array)
```

Modified Array: [0 0 0 25 30]

The combined **mask (array > 15) & (array < 30)** selects elements that are greater than 15 and less than 30.

Applying the mask returns the elements **[20, 25]**.

```
import numpy as np

# Create a numpy array
array = np.array([10, 15, 20, 25, 30])

# Create masks for elements greater than 15 and less than 30
mask = (array > 15) & (array < 30)

# Apply the combined mask
filtered_elements = array[mask]

print("Combined Mask:", mask)
print("Filtered Elements with Combined Mask:", filtered_elements)
```

Combined Mask: [False False True True False]
Filtered Elements with Combined Mask: [20 25]